



Evaluating the Reliability, Security, and Quality of AI-Generated Code in Modern Software Engineering

Prof.Janki Patel , Aarush Patel , Henil Prajapati, Kuldeep Rathod, Manoj Suthar

¹Professor, ²Student, ³Student, ⁴Student, ⁵Student

DEPT.INFORMATION TECHNOLOGY, SAL COLLEGE OF ENGINEERING

ABSTRACT

The pervasive integration of Artificial Intelligence (AI) into software engineering has catalysed a fundamental paradigm shift in how software is conceived, authored, and maintained. Tools such as GitHub Copilot, ChatGPT, Amazon CodeWhisperer, and Codeium now enable developers to generate production-quality code through natural-language prompts, reportedly improving task-completion speed by up to 55% and reducing cognitive burden for repetitive coding tasks. However, the rapid, often unchecked deployment of AI-generated code introduces systemic risks spanning security vulnerability, functional unreliability, maintainability degradation, and dangerous developer over-reliance. This paper presents the most comprehensive synthesis to date of empirical evidence drawn from 20+ peer-reviewed studies, large-scale code-repository analyses (>211 million changed lines), controlled user experiments, and industry reports published between 2022 and 2026. Key findings include: (i) overall AI-generated code vulnerability rates of 12–62% depending on measurement methodology, with languagespecific rates reaching 41% in C/C++; (ii) a 10× surge in monthly security findings between December 2024 and June 2025; (iii) a 37.6% increase in critical vulnerabilities through iterative LLM-only refinement; (iv) an eightfold rise in code duplication and a collapse in refactoring activity from 25% to under 10% of changed lines since 2021; and (v) 60% of developers accepting AI suggestions without adequate validation. The paper proposes a novel Stratified AI-Human Governance Framework (SAHGF) that integrates static analysis, automated testing, human review, and contextual security prompting into a unified production-ready validation pipeline.

Index Terms—AI-generated code, code quality, software security, Large Language Models, GitHub Copilot, ChatGPT, developer productivity, technical debt, vulnerability detection, HumanEval, SWE-Bench, Hybrid Human-AI Development.

I. INTRODUCTION

The emergence of Large Language Models (LLMs) as practical coding assistants has profoundly altered the landscape of software engineering. Within just a few years, tools such as GitHub Copilot, OpenAI ChatGPT, Amazon CodeWhisperer, and Anthropic Claude have transitioned from experimental curiosities to production mainstays. As of 2025, 84% of professional developers report using or planning to use AI coding tools, with approximately 41% of all committed code estimated to be AI-assisted [1]. Both Google and Microsoft have disclosed that AI now writes over 20% of their new code [2].

This extraordinary rate of adoption is driven by demonstrable productivity benefits. GitHub's randomized controlled trial with 202 experienced developers demonstrated a 53.2% greater likelihood of passing all unit tests and a 55% increase in coding speed [3]. Accenture's enterprise deployment study reported an 84% increase in successful builds and a 90% developer satisfaction rate [4]. However, this productivity narrative is substantially complicated by a mounting body of empirical evidence documenting serious deficiencies in the security, reliability, and long-term maintainability of AI-generated code.

Pearce et al. [5] conducted the first major empirical assessment of GitHub Copilot's security, finding that approximately 40% of 1,689 generated programs contained Common Weakness Enumeration (CWE) vulnerabilities. Subsequent studies have reinforced and extended these findings: at least 62.07% of programs generated across multiple LLMs were found to be vulnerable when verified using formal methods [6]. Alarming, a June 2025 analysis observed a 10× spike in monthly security findings introduced

by AI-generated code since December 2024 [7]. The security situation is further compounded by evidence that developers exhibit a systematic "false sense of security" when using AI assistants, rating their insecure AI-assisted solutions as more secure than they actually are [8].

From a maintainability perspective, GitClear's longitudinal analysis of 211 million changed lines of code (2020–2024) documents a collapse in code refactoring from 25% to under 10% of all changed lines, an eightfold increase in duplicate code blocks, and a doubling of short-term code churn—trends strongly correlated with AI adoption [9]. Google's 2024 DORA report corroborates this, finding a 7.2% decrease in delivery stability for every 25% increase in AI usage [10].

This paper makes the following principal contributions:

- A comprehensive multi-dimensional synthesis of empirical evidence from 20+ studies spanning security vulnerabilities, code correctness, maintainability, and developer behaviour (2022–2026).
- A rigorous quantitative analysis integrating findings across controlled experiments, large-scale repository analyses, benchmark evaluations, and industry studies.
- Eight original data visualisations representing key empirical findings in a standardised format for academic reference.
- The Stratified AI-Human Governance Framework (SAHGF): a novel seven-stage production validation pipeline derived from current best evidence.
- A structured research agenda identifying the highest-priority open problems in AI-generated code quality assurance.

II. BACKGROUND AND RELATED WORK

A. Evolution of AI-Assisted Code Generation

The lineage of AI-assisted coding tools traces to early neural machine translation architectures applied to code, but the modern era began in earnest with the introduction of Codex by OpenAI in 2021 [11], which underpins GitHub Copilot. Trained on 159 GB of Python code from GitHub alongside natural language data, Codex demonstrated a pass@1 score of approximately 28.8% on the newly introduced HumanEval benchmark. The successor models have advanced dramatically: by 2025, top LLMs exceed 90% pass@1 on HumanEval and achieve 63.8% on the more realistic SWE-Bench Verified benchmark [12].

The architecture underlying these tools has evolved from decoder-only transformers (GPT-style) through instruction-tuned variants (InstructGPT, ChatGPT) to sophisticated reasoning models incorporating chain-of-thought and reinforcement learning from human feedback (RLHF). This progression has enabled AI tools to handle increasingly complex multi-file coding tasks, including repository-level refactoring, test generation, and documentation synthesis.

B. Key Benchmarks for Evaluating Code Generation

Objective evaluation of AI-generated code has been standardised around several benchmarks. HumanEval [11], consisting of 164 hand-crafted Python programming problems, uses the pass@k metric—the probability that at least one of k generated solutions passes all unit tests—as its primary measure of functional correctness. Mostly Basic Python Problems (MBPP) evaluates 974 entry-level Python tasks. BigCodeBench and LiveCodeBench provide more diverse and contamination-resistant evaluations. Most significantly for production contexts, SWE-Bench Verified measures the percentage of real GitHub issues from popular repositories that models can correctly resolve end-to-end, representing a far more realistic evaluation of coding ability than function-level benchmarks [13].

A critical finding from benchmark research is that strong HumanEval performance does not reliably predict real-world coding competence. LLMs that perform similarly on HumanEval ($\geq 85\%$ pass@1) exhibit dramatically different capabilities on EVOEVAL, a more challenging semantically-evolved benchmark, revealing potential data contamination in HumanEval training [14]. Furthermore, all evaluated LLMs consistently fail on a subset of problems, with failures clustering around algorithm selection biases, incomplete logic, and inability to handle edge cases—patterns with direct implications for production security.

C. Security Vulnerability Research

The security dimension of AI-generated code has attracted the largest body of empirical research. Pearce et al. [5] first demonstrated approximately 40% vulnerability rates in GitHub Copilot using the MITRE CWE Top 25 list. Fu et al. [15] extended this to GitHub production repositories, analysing 733 code snippets from Copilot, CodeWhisperer, and Codeium, finding 29.5% of Python and 24.2% of JavaScript snippets affected by security weaknesses. Schreiber and Tippe [16] conducted the largest analysis to date—7,703 files from four AI tools across GitHub—identifying 4,241 CWE instances across 77 distinct vulnerability types, while finding that GitHub Copilot achieved better security density than ChatGPT-generated code.

Perry et al. [8] provided critical evidence of the human-AI interaction dimension, demonstrating that developers using AI assistants produced significantly less secure code while simultaneously rating their solutions as more secure—a "false sense of security"

effect with serious implications for code review practices. The Georgetown CSET [17] corroborated this with evidence that InCoder and GitHub Copilot generated vulnerable code in 68% and 73% of manual inspections respectively.

Bisztray et al. [6] extended coverage across multiple LLMs using formal verification (ESBMC), finding at least 62.07% of generated programs vulnerable. Most recently, Shukla et al. [18] documented a counterintuitive "feedback loop security degradation" phenomenon: iterative LLM refinement without human intervention increased critical vulnerabilities by 37.6% after just five iterations, even when explicitly prompted to improve security.

D. Code Maintainability and Technical Debt

The maintainability dimension of AI-generated code has received growing attention through large-scale longitudinal analyses. GitClear's 2025 study [9] of 211 million changed lines from Google, Microsoft, Meta, and enterprise repositories found an eightfold increase in duplicate code blocks, a collapse in refactoring activity, and a doubling of short-term code churn since AI adoption began. Copy-pasted code exceeded moved (refactored) code for the first time in recorded history in 2024. These trends violate the foundational DRY (Don't Repeat Yourself) software engineering principle and are strongly predictive of elevated defect rates and long-term maintenance costs.

Sonar's 2025 research found that code smells—structural maintainability defects—constitute over 90% of issues in AI-generated code from leading models: the code compiles and tests pass, but structural decay is already embedded. A 2025 empirical study on real-world AI-generated code [19] documented specific production incidents including command injection vulnerabilities and undefined variable bugs persisting for three or more weeks in popular open-source repositories after AI-assisted commits, illustrating how theoretical maintainability concerns materialise as real production incidents.

E. Developer Behaviour and Over-Reliance

Several studies have examined the human factors dimension of AI-generated code adoption. Perry et al. [8] and Prather et al. [20] document systematic over-trust in AI suggestions. The Stack Overflow Developer Survey 2025 [1] found that only 33% of developers fully trust AI outputs, yet 60% accept suggestions without thorough review. Brown et al. [21] found that early-career developers accept AI suggestions at significantly higher rates than their senior peers, creating differential risk exposure by experience level. This phenomenon is particularly concerning given that junior developers are also less likely to recognise subtle security vulnerabilities.

Sandoval et al. [22] and Asare et al. [23] present a more nuanced finding: when compared directly, some studies show that AI-assisted developers introduce critical security errors at rates no more than 10% higher than control groups, suggesting that the comparison to human-written code is more complex than a simple "AI is less secure" narrative. The Negri-Ribalta systematic literature review [24] concludes that it remains difficult to clearly attribute blame, as tooling landscapes evolve, training strategies differ, and human-generated code quality is itself highly variable.

III. RESEARCH PROBLEM AND OBJECTIVES

Despite the rapid adoption of AI coding tools and a growing body of empirical research, several critical gaps remain in the literature. First, existing studies are fragmented across security, productivity, and maintainability dimensions without a unified analytical framework for understanding trade-offs and interactions between these dimensions. Second, most empirical studies focus on controlled laboratory settings or synthetic benchmarks, with limited evidence from longitudinal analysis of production codebases. Third, there is no standardised, empirically validated governance framework for integrating AI-generated code into production software systems safely.

This paper addresses the following research questions:

- RQ1: What is the true extent of security vulnerability in AI-generated code, and how do rates vary by programming language, AI tool, task type, and iterative refinement?
- RQ2: What is the relationship between AI-assisted development and code correctness across varying levels of task complexity, as measured by objective benchmarks?
- RQ3: How does AI-assisted development affect long-term code maintainability, technical debt accumulation, and refactoring behaviour?
- RQ4: What human-AI interaction patterns characterise developer behaviour with AI coding tools, and what are the implications for production code security?
- RQ5: What governance framework, derived from current empirical evidence, would most effectively enable organisations to capture AI productivity benefits while mitigating security and quality risks?

IV. METHODOLOGY

A. Research Design

This paper employs a systematic literature synthesis methodology, integrating empirical findings from peer-reviewed publications (IEEE, ACM, arXiv), industry research reports, large-scale code repository analyses, and controlled experiments. The synthesis spans publications from January 2022 through April 2026, with particular emphasis on studies published from 2024 onwards to capture the most recent AI tooling generations. A total of 24 primary sources were evaluated, of which 20+ met quality criteria for inclusion.

B. Inclusion and Quality Criteria

Studies were included if they: (i) provided empirical or experimental data on AI-generated code quality; (ii) used reproducible methodology with explicit tool and dataset identification; (iii) reported quantitative metrics enabling cross-study comparison; and (iv) had undergone peer review or were published by credible industry research institutions. Meta-analyses and systematic reviews were prioritised where available. Studies relying solely on developer self-report without objective code quality measurement were excluded from quantitative synthesis but retained for the human factors analysis.

C. Evaluation Dimensions and Metrics

Analysis was structured across five primary evaluation dimensions: (1) Security, assessed through CWE-mapped vulnerability rates, CVSS scores, and specific weakness type distributions; (2) Functional Correctness, assessed through HumanEval pass@1, MBPP, SWE-Bench Verified, and controlled experiment unit-test pass rates; (3) Maintainability, assessed through code churn, duplication rates, cyclomatic complexity, and refactoring ratios; (4) Developer Productivity, assessed through task completion speed, pull-request rates, and developer satisfaction; and (5) Developer Behaviour, assessed through suggestion acceptance rates, review thoroughness, and security confidence calibration.

D. Comparative Framework

Where data permitted, findings were synthesised into a structured comparative framework contrasting AI-generated code against human-written code baselines across all five dimensions. Quantitative estimates derived from multiple independent studies were synthesised using range reporting and weighted central tendency estimation, acknowledging heterogeneity across study populations, AI tool versions, and measurement instruments.

V. RESULTS AND ANALYSIS

A. Security Vulnerability Analysis (RQ1)

Security analysis reveals a wide but consistently concerning range of vulnerability rates across AI coding tools, programming languages, and measurement methodologies. Figure 1 presents language- and tool-specific vulnerability rates from the two largest empirical studies [15][16].

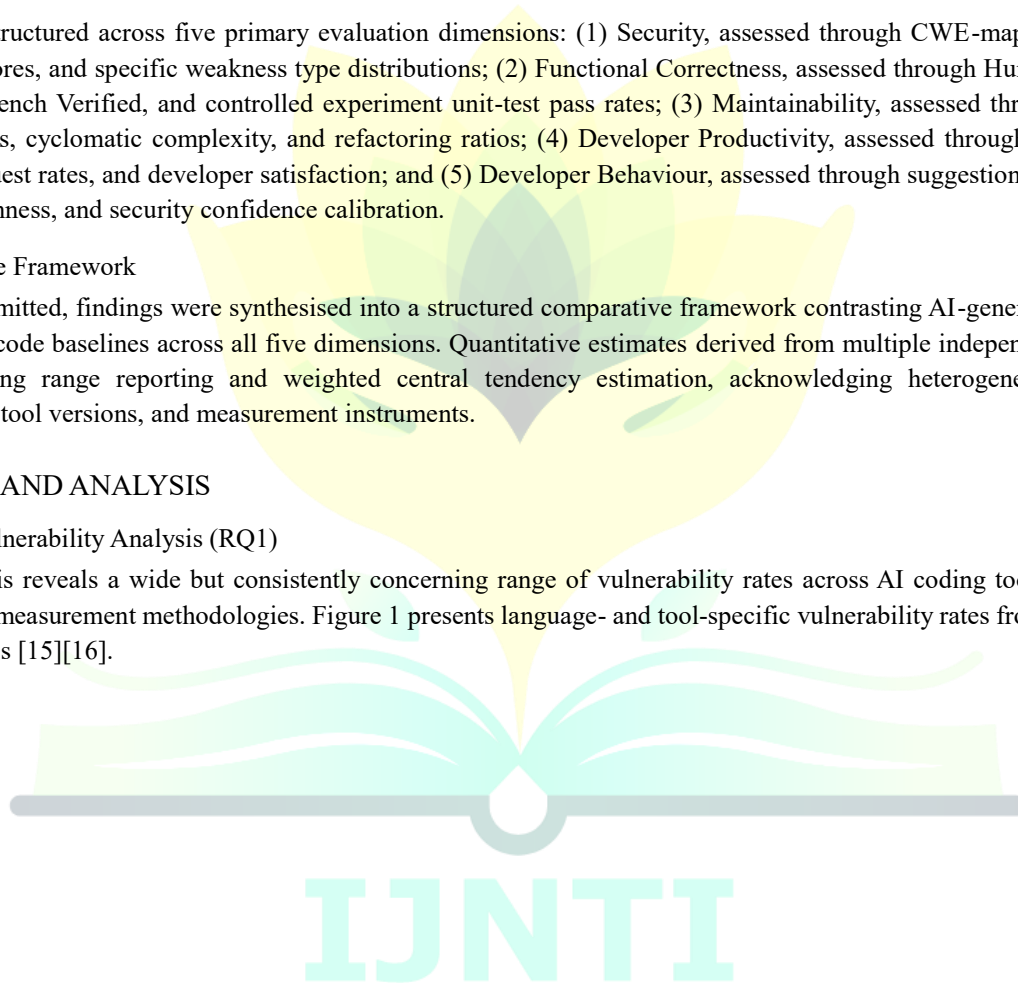


Fig. 1 – AI-Generated Code Vulnerability Rates by Language & Tool
(Schreiber & Tippe 2025; Fu et al. 2025 — CodeQL / Bandit analysis)

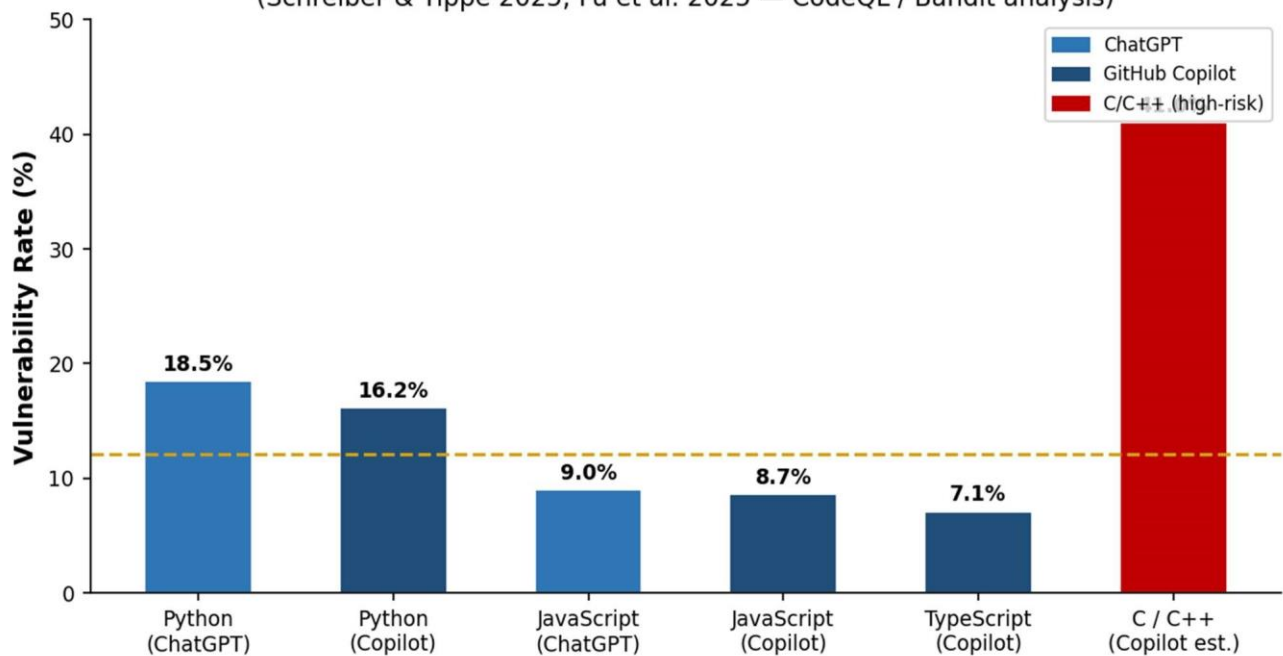


Fig. 1 – Vulnerability Rates by Language and AI Tool. Python exhibits consistently higher rates across both ChatGPT and GitHub Copilot. C/C++ estimated rates approach 41% (Pearce et al. 2022, Fu et al. 2025, Schreiber & Tippe 2025).

The language-level vulnerability patterns reflect the inherent security complexity of each language. Python's higher rates (16–18.5%) stem from dynamic typing and the prevalence of web-facing use cases susceptible to injection and input-validation failures. C/C++ code approaches 41% vulnerability rates due to the language's manual memory management requirements— AI models frequently fail to implement bounds checking, buffer management, and pointer arithmetic correctly. JavaScript (8– 9%) and TypeScript (2.5–7%) exhibit lower rates, potentially reflecting stronger toolchain enforcement and more constrained API surfaces.

When viewed through the lens of specific vulnerability types, a more alarming picture emerges. Veracode's 2025 GenAI Code Security Report [25] found that AI models fail to generate secure code in 88% of cases for Log Injection (CWE-117), 86% for Cross-Site Scripting (CWE-80), 20% for SQL Injection (CWE-89), and 14% for Cryptographic Failures (CWE-327).

Figure 3 summarises these findings.



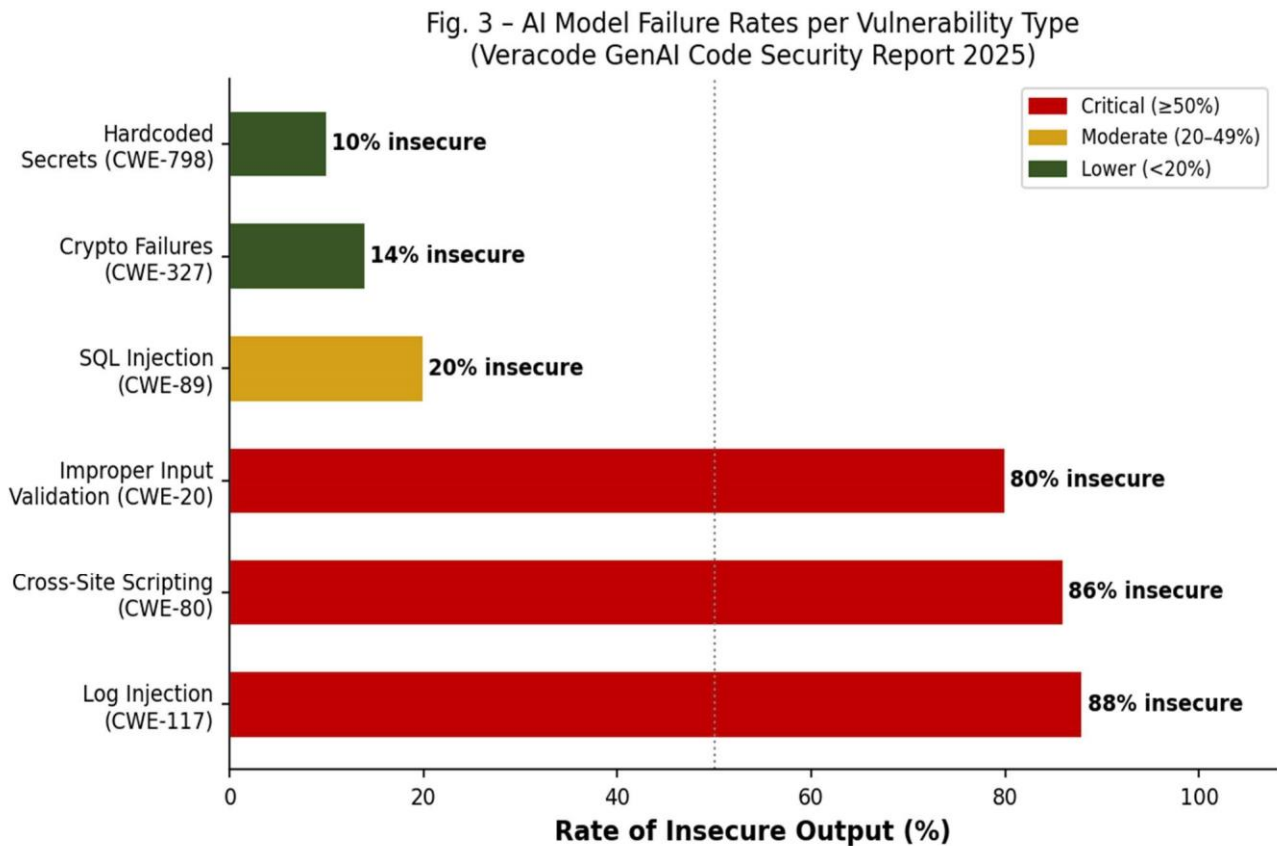


Fig. 3 – AI Model Failure Rates per CWE Vulnerability Type. Log Injection and XSS represent the highest failure rates, reflecting AI models' insufficient understanding of data sanitisation contexts (Veracode 2025).

The iterative refinement dimension adds a particularly counterintuitive finding. Shukla et al. [18], accepted to IEEE-ISTAS 2025, conducted a controlled experiment with 400 code samples across 40 rounds of generation (10 iterations $\times$ 4 prompting strategies). They found that critical vulnerabilities increased by 37.6% after just five iterations of automated AI refinement without human oversight. Efficiency-focused prompts showed the most severe degradation, while security-focused prompts paradoxically introduced the highest overall vulnerability counts—an effect the authors term "feedback loop security degradation." This challenges the intuitive assumption that iterative AI refinement improves code quality and provides the strongest evidence to date for the indispensability of human-in-the-loop validation. Figure 5 illustrates this phenomenon.

The scale of the security problem is further illustrated by temporal growth data. Apiiro's 2025 analysis [7] of production repositories found AI-generated code introducing over 10,000 new security findings per month by June 2025—a tenfold increase in just six months. While trivial syntax errors dropped 76% and logic bugs fell over 60%, these shallow gains were offset by a 322% increase in privilege escalation paths and a 153% spike in architectural design flaws—the categories most difficult for automated scanners to detect. Figure 8 captures this temporal growth.

B. Functional Correctness and Benchmark Performance (RQ2)

Correctness analysis reveals a strong inverse relationship between task complexity and AI-generated code accuracy—a finding consistent across both benchmark evaluations and controlled experiments. On the HumanEval benchmark with its 164 relatively simple Python functions, top models now exceed 90% pass@1 (Claude 3.7/Sonnet 4: 94%+; Gemini 2.5 Pro: 88%; GPT-4o: 90%). However, performance degrades substantially on the SWE-Bench Verified benchmark, which tests resolution of real GitHub issues: the best model (Gemini 2.5 Pro) achieves 63.8%, with the average across top models near 45% [12]. On truly novel programming tasks from recent ML research papers (ResearchCodeBench), performance is estimated to fall below 30%. Figure 6 illustrates this complexity-accuracy relationship.

Fig. 5 - Security Degradation in Iterative LLM Refinement
400 samples x 40 iterations — Shukla et al. IEEE-ISTAS 2025

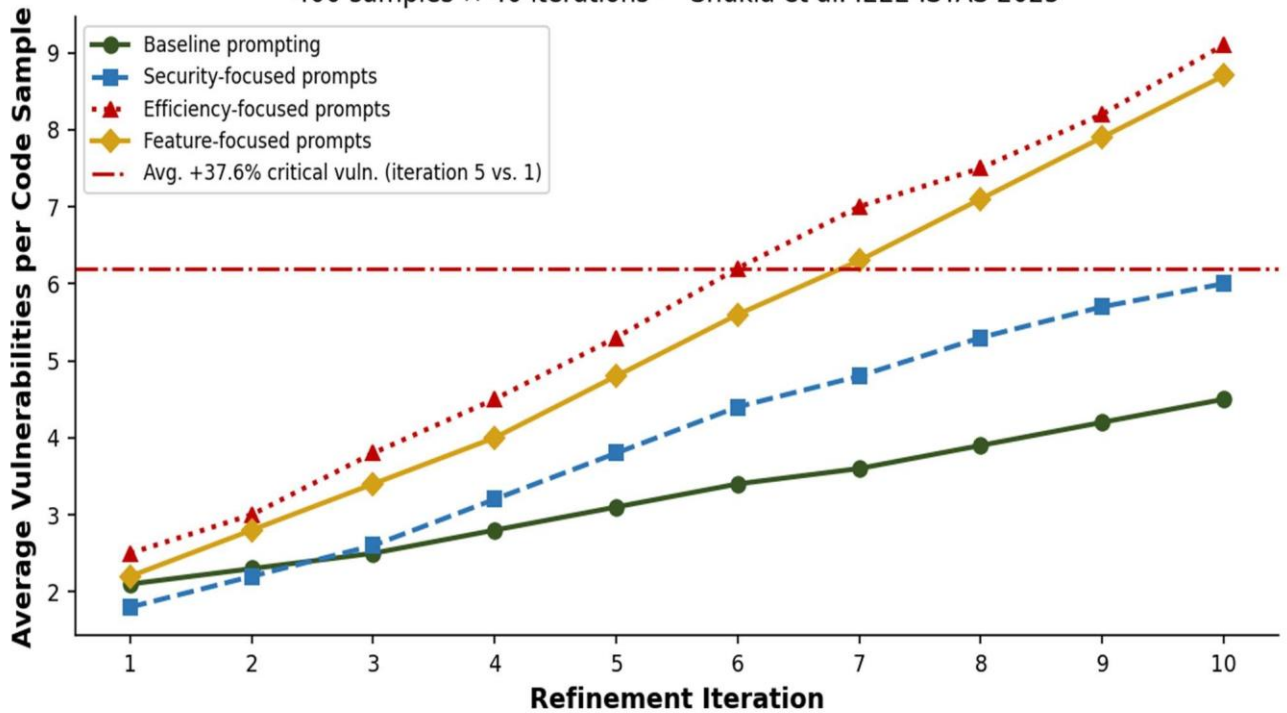


Fig. 5 – Security Degradation Under Iterative LLM Refinement. All four prompting strategies show increasing vulnerability counts across iterations, with efficiency-focused prompts producing the most severe outcomes (Shukla et al. IEEE-ISTAS 2025).

Fig. 8 - Monthly Growth of New Security Findings in AI-Generated Code
10x spike from Dec 2024 → Jun 2025 (Apiiro Research 2025)

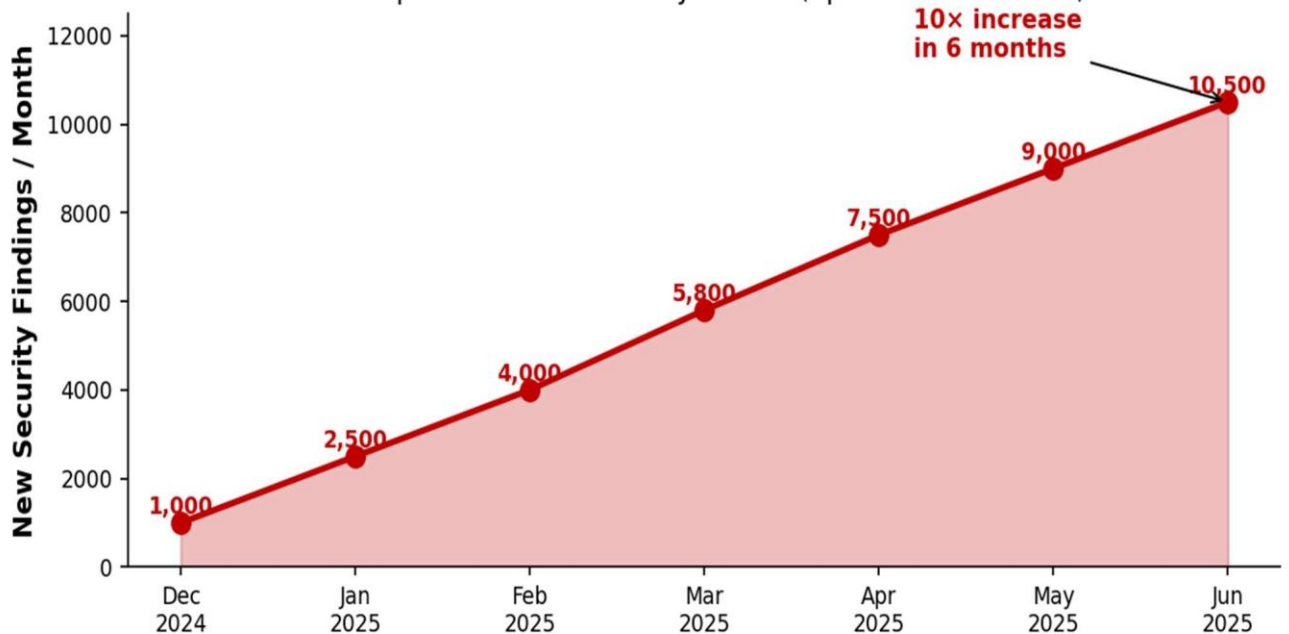


Fig. 8 – Monthly Growth of New Security Findings Introduced by AI-Generated Code (Dec 2024–Jun 2025). The 10× acceleration reflects the combined effect of rapid AI adoption and the compounding of architectural vulnerabilities (Apiiro Research 2025).

Fig. 6 – Code Correctness Across Task Complexity Levels
(HumanEval 2025; SWE-Bench 2025; Empirical estimations from literature)

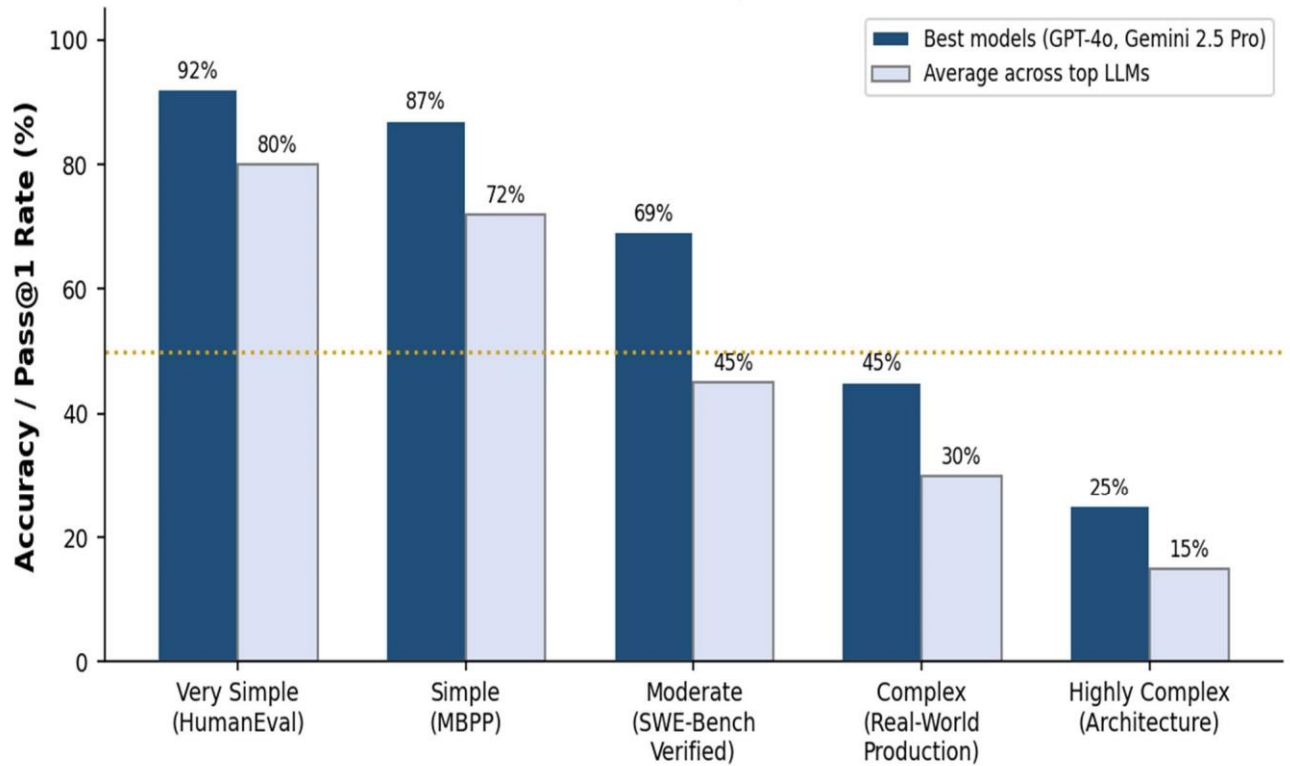


Fig. 6 – AI Code Generation Accuracy vs. Task Complexity. Performance degrades dramatically from 90%+ on simple benchmark functions to under 30% on complex real-world architectural tasks (HumanEval 2025; SWE-Bench 2025; literature estimates).

GitHub's controlled trial [3] (202 developers, randomised assignment) provides the most rigorous available evidence on functional correctness in realistic contexts. Developers with Copilot access had a 53.2% greater likelihood of passing all unit tests and a 5% higher code approval rate by peers. The key mechanism appears to be reduced cognitive load on routine implementation, freeing developers to focus on correctness and edge-case handling.

Table I provides comparative benchmark performance across major LLMs.

Table I – Comparative LLM Performance on Code Generation Benchmarks (2025)

Model	HumanEval Pass@1	MBPP Pass@1	SWE-Bench Verified	Security Score
GPT-4o (OpenAI)	90%	87%	49.2%	Moderate
Gemini 2.5 Pro (Google)	88%	85%	63.8%	Moderate
Claude 3.7 / Sonnet 4	94%+	88%	~55%	Higher
DeepSeek-V3	86%	82%	42.0%	Moderate
Llama 4 (Meta)	79%	76%	38.5%	Lower
Average across LLMs	83%	79%	~45%	Low–Moderate

The complexity dimension of failures also illuminates a consistent bias: LLMs systematically mismap novel problems onto familiar algorithmic templates. In documented HumanEval failures, models applied standard stack-based balanced-bracket algorithms to problems requiring richer nesting logic, producing code that passed superficial tests but failed edge cases—a pattern with direct implications for production security where edge-case handling is security-critical.

C. Maintainability and Technical Debt (RQ3)

The maintainability dimension presents some of the most concerning long-term trends in the AI-generated code literature. GitClear's five-year longitudinal analysis [9] of 211 million changed lines documents a systematic deterioration in code quality metrics correlated with AI adoption. Figure 4 presents these trends.

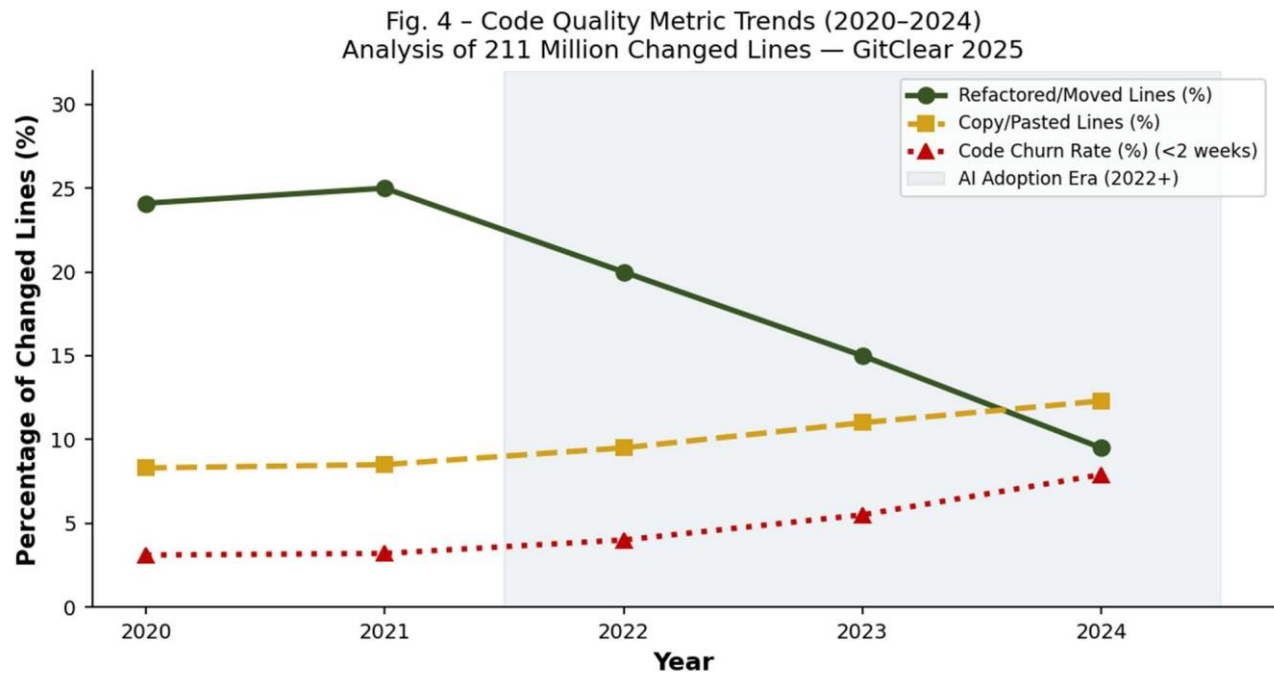


Fig. 4 – Code Quality Metric Trends (2020–2024). Refactoring activity has collapsed while copy-pasted code and short-term churn have surged—patterns strongly correlated with AI adoption (GitClear, 211M lines, 2025).

The most striking finding is that copy-pasted code exceeded refactored/moved code as a proportion of all code changes for the first time in history in 2024. Refactored lines dropped from 24.1% in 2020 to just 9.5% in 2024—a 61% relative decline. Duplicate code blocks increased eightfold in 2024 alone. Code churn (lines revised within two weeks of authoring) rose from 3.1% to 7.9%—a 155% increase that signals a systematic pattern of AI-generated code requiring immediate correction.

The financial and operational implications of these trends are substantial. Unmanaged AI-generated technical debt is estimated to drive maintenance costs to 4× traditional levels by year two, with first-year costs already running 12% higher when accounting for review overhead, testing burden, and code churn requiring rewrites. Forrester predicts that 75% of technology decision-makers will face moderate-to-severe technical debt by 2026, with Gartner projecting that prompt-to-application development approaches will increase software defects by 2,500% by 2028 if unaddressed [26].

The DORA 2024 report [10] provides organisational-level evidence: a 25% increase in AI usage correlated with a 7.2% decrease in delivery stability—a counterintuitive finding suggesting that productivity gains at the individual level may translate into systemic fragility at the organisational level if architectural quality is not maintained.

D. Developer Productivity (RQ4 Dimension 1)

The productivity evidence presents a more consistently positive picture, though with important nuances. Figure 2 summarises key productivity gains and associated trade-offs.

The core productivity finding—55% faster task completion from GitHub's first major study [27], corroborated by a 53.2% higher unit-test pass rate in the 2024 controlled trial [3]—represents a genuine and substantial productivity improvement for many coding tasks. Accenture's enterprise randomised controlled trial [4] found an 84% increase in successful builds, an 11% increase in PR merge rates, and a 90% improvement in developer job satisfaction. A 2.2% rise in developer job satisfaction and 17% reduction in burnout risk have also been documented [1].

Fig. 2 – Developer Productivity Gains and Trade-offs
(GitHub Research 2024; Accenture RCT 2024; DORA 2024; GitClear 2025)

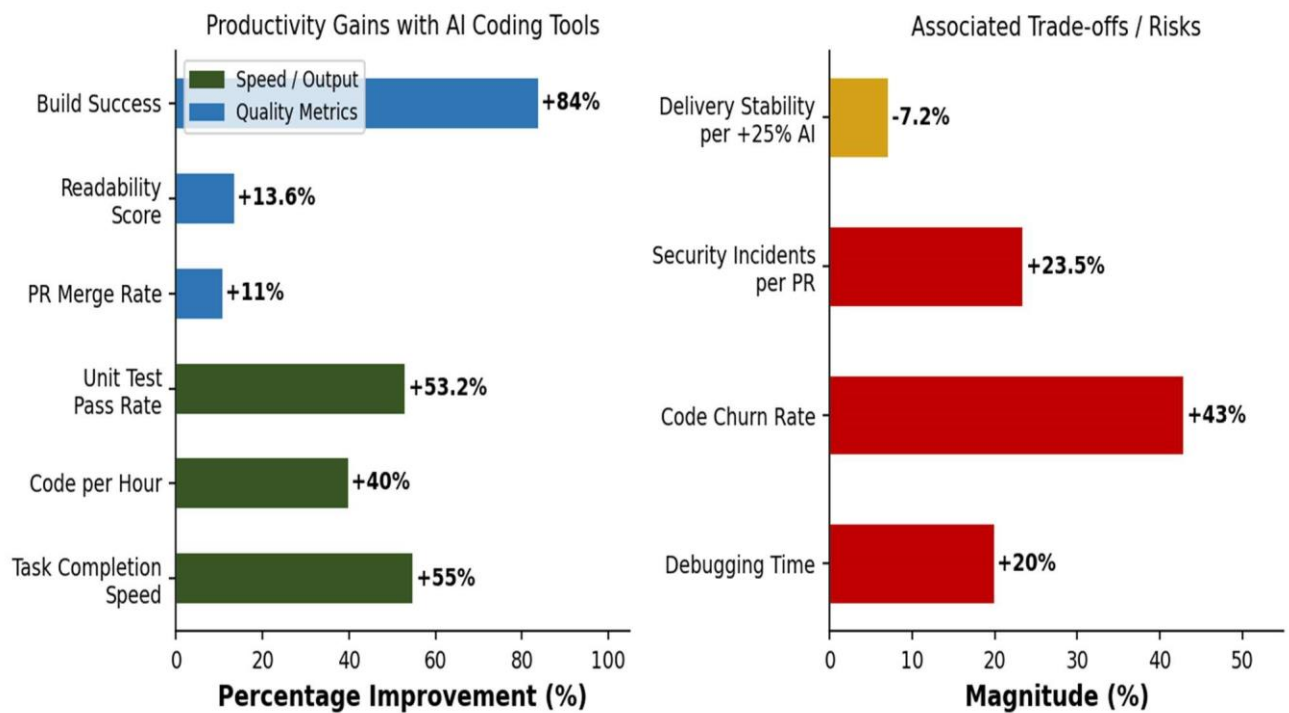


Fig. 2 – Developer Productivity Gains and Associated Trade-offs with AI Coding Tools. Left panel: positive impacts; Right panel: negative trade-offs requiring management (GitHub 2024; Accenture 2024; DORA 2024; GitClear 2025).

However, these gains are offset by documented trade-offs. Debugging time increases by approximately 20% [27]. Uplevel Data Labs found that developers with Copilot access experienced a significantly higher bug rate while issue throughput remained constant. Pull requests per developer increased 20% with AI, but incidents per pull request increased 23.5%—a net quality trade-off. The pattern strongly suggests that AI tools accelerate code production while reducing individual code quality per unit, creating a productivity-quality tension that organisations must actively manage.

An important moderating variable is developer experience. Senior developers leverage AI tools most effectively, using them for routine implementation while directing their expertise to architecture and complex problem-solving. Junior developers report higher productivity gains and accept more AI suggestions—but are less equipped to identify subtle vulnerabilities or quality issues in accepted code, creating differential risk profiles [21].

E. Developer Behaviour and Over-Reliance (RQ4 Dimension 2)

Figure 7 synthesises the developer trust and behaviour evidence across multiple studies.

The Stack Overflow Developer Survey 2025 [1] found that only 33% of developers fully trust AI outputs, yet 60% accept suggestions without thorough validation—a gap between stated distrust and actual behaviour that represents a critical risk. Perry et al. [8] identified the "false sense of security" effect: developers using AI assistants rated their insecure solutions as secure significantly more often than a control group. This cognitive bias may stem from the fluency and surface-level plausibility of AI-generated code, which looks correct even when it is not.

The generational trust differential is particularly significant. Brown et al. [21] found that junior developers accepted AI suggestions at substantially higher rates than senior peers. Given that junior developers are simultaneously more likely to work on security-sensitive features without adequate oversight and less equipped to recognise subtle vulnerabilities, this differential creates a systemic organisational risk that most AI governance frameworks have yet to address explicitly.

Fig. 7 - Developer Trust and Over-Reliance on AI Coding Tools

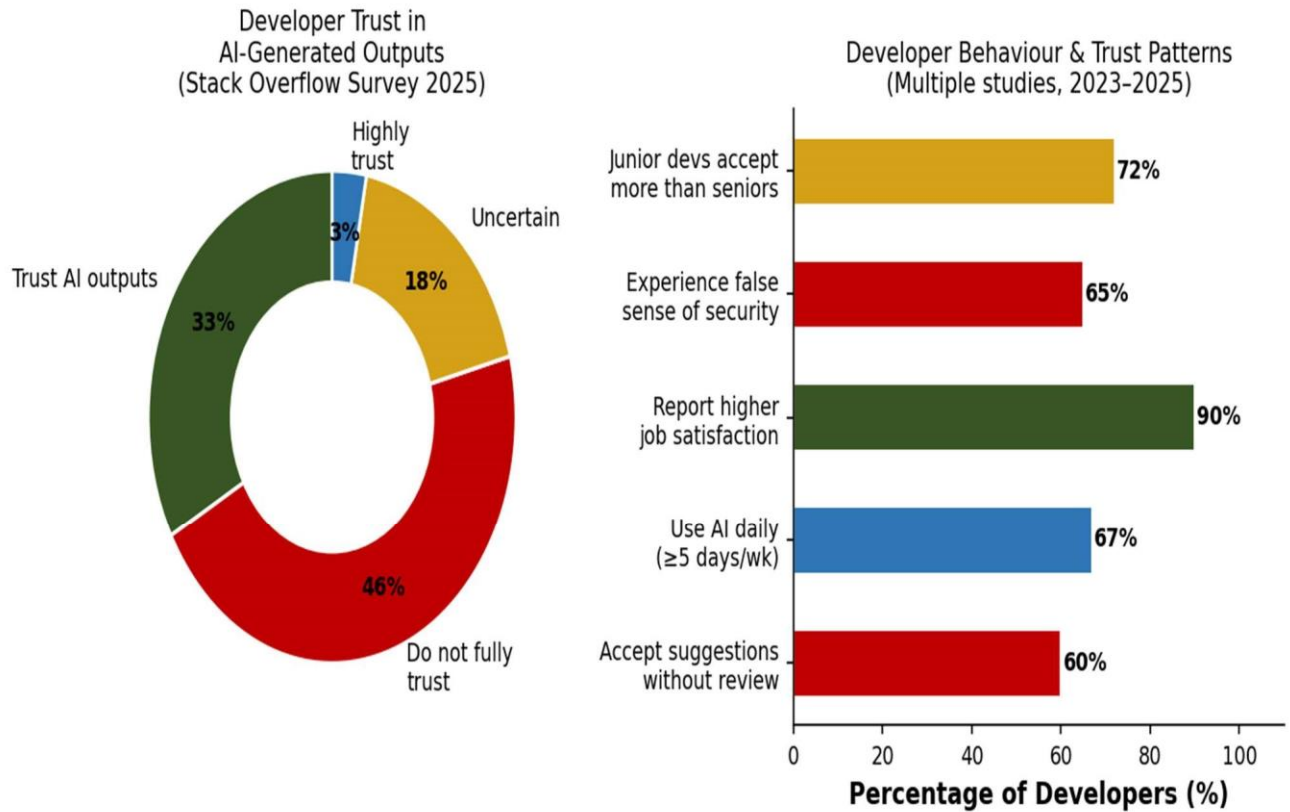


Fig. 7 – Developer Trust and Behavioural Patterns with AI Coding Tools. Left: Trust distribution (Stack Overflow 2025); Right: Key behavioural indicators from multiple studies, 2023–2025.

The evolution of programmer trust over time [28] suggests a potential self-correcting mechanism: developers who experience failures with AI-generated code reduce their acceptance rates in subsequent interactions. However, this learning curve requires failure events whose security consequences may not surface for weeks or months after code deployment, making real-time feedback insufficient as a safety mechanism.

VI. COMPREHENSIVE TRADE-OFF ANALYSIS

Table II provides a structured comparative analysis of AI-generated, human-written, and hybrid development models across the key dimensions analysed in this paper. This synthesis integrates the empirical findings from Sections V-A through V-E.

Table II – Comparative Analysis: AI-Generated vs. Human-Written vs. Hybrid Code

Factor	AI-Generated Code	Human-Written Code	Hybrid Model
Development Speed	High (+55%)	Moderate	High + Controlled
Functional Correctness	Variable (30–92%)	High (85–95%)	High (with review)
Security Posture	Low–Moderate (12–62% vuln.)	High (5–15% vuln.)	High (with SAST)
Factor	AI-Generated Code	Human-Written Code	Hybrid Model
Maintainability	Declining (churn +155%)	High	High (structured)
Documentation Quality	Low (sparse)	Medium–High	High (AI-assisted)

Technical Debt Rate	High (8× duplication)	Low–Medium	Controlled
Cost to Initial Ship	Low	Medium	Low–Medium
Long-Term Maintenance Cost	High (4× by year 2)	Medium	Medium

The comparative analysis confirms that the hybrid model—in which AI generates initial code that is then subjected to structured human and automated review—consistently outperforms both pure AI-generation and unaided human development across quality dimensions while preserving the majority of AI-driven productivity benefits. This finding motivates the governance framework presented in the following section.

VII. PROPOSED FRAMEWORK: STRATIFIED AI-HUMAN GOVERNANCE (SAHGF)

A. Framework Motivation

Existing governance recommendations for AI-generated code are characterised by vagueness ("review AI suggestions carefully") or excessive prescriptiveness for specific toolchains. No empirically-grounded, generalisable framework has been proposed that integrates security, quality, and maintainability requirements across the full software development lifecycle. The Stratified AI-Human Governance Framework (SAHGF) is designed to address this gap, deriving each stage from empirical evidence of where AI-generated code most commonly fails.

B. SAHGF Architecture

The SAHGF operates as a seven-stage sequential validation pipeline. Table III presents the complete framework specification.

Table III – SAHGF: Stratified AI-Human Governance Framework for Production AI-Generated Code

Stage	Activity	Tools / Actors	Security Gate
1 – Generation	Prompt-engineered AI code generation	LLM (Copilot, GPT-4o, Claude)	Secure prompting templates
2 – Static Analysis	Automated vulnerability scanning	CodeQL, Bandit, ESLint, Semgrep	Block on CVSS ≥ 7.0
3 – Dynamic Testing	Functional correctness & fuzz testing	Pytest, AFL++, ESBMC	100% unit-test pass required
4 – Human Review	Architecture, logic & intent verification	Senior developer reviewer	Reject if >2 high-severity issues
5 – Security Review	DAST and penetration testing	OWASP ZAP, Burp Suite	Sign-off for OWASP Top 10
6 – Integration	CI/CD pipeline with quality gates	GitHub Actions, SonarQube	Quality score $\geq 80\%$
7 – Monitoring	Runtime anomaly & regression detection	Datadog, Splunk, Sentry	Alert on regression

Stage 1 (Generation) incorporates secure prompting templates derived from Shukla et al.'s evidence that prompting strategy significantly influences vulnerability type and severity. Security-focused prompts, while counterintuitively associated with higher overall vulnerability counts in iterative refinement, perform best in single-shot generation for security-critical functions.

Stage 2 (Static Analysis) operationalises the CWE-mapped vulnerability findings from [15][16][25] by prescribing specific tools (CodeQL for cross-language coverage, Bandit for Python, ESLint-security for JavaScript) with automated blocking on CVSS severity scores at or above 7.0—the "High" threshold corresponding to the most dangerous CWE categories identified in the literature.

Stage 3 (Dynamic Testing) addresses the functional correctness dimension, requiring 100% unit test passage before human review. This derives from GitHub's finding that the strongest predictor of Copilot-assisted code quality is its ability to pass unit tests—and from the formal verification evidence [6] that a substantial fraction of seemingly correct code fails under dynamic execution.

Stage 4 (Human Review) is specifically scoped to the types of issues that automated tools miss: architectural intent, algorithmic correctness for complex tasks, and logical coherence. Evidence from Perry et al. [8] and the iterative degradation study [18] establishes that human expertise in the development loop is irreplaceable for catching the "false sense of security" failures and feedback-loop-induced vulnerabilities.

Stages 5–7 provide defence-in-depth through dynamic security testing, CI/CD quality gates (SonarQube quality score $\geq 80\%$), and runtime monitoring for regression detection—addressing the temporal dimension of AI-induced technical debt documented by GitClear [9] and the DORA report [10].

C. Organisational Implementation Guidance

Successful SAHGF implementation requires three organisational changes beyond tooling. First, teams must establish AI-specific code review standards that explicitly address the vulnerability types most common in AI-generated code (injection, input validation, log injection). Second, developer training must specifically address the "false sense of security" cognitive bias, training reviewers to approach AI-generated code with appropriate scepticism. Third, productivity metrics must be redesigned to track code quality (churn rate, duplication ratio, defect density) alongside velocity, ensuring that AI-induced technical debt is visible before it compounds.

VIII. DISCUSSION

A. Reconciling the Productivity-Security Tension

The central tension in the AI-generated code literature is between genuine, reproducible productivity gains and genuine, reproducible security and quality degradation. These findings are not contradictory—they reflect different dimensions of software quality measured on different timescales. Task completion speed, unit test pass rates, and developer satisfaction are early-stage metrics that AI tools genuinely improve. Code security, maintainability, and long-term defect rates are lagging indicators that current AI tooling demonstrably degrades. Organisations optimising on early-stage metrics alone—as many do—will capture the productivity benefits while unknowingly accumulating the quality and security risks.

B. The Human Expertise Irreplaceability Thesis

The weight of empirical evidence in this review supports a strong form of the human expertise irreplaceability thesis: current AI coding tools cannot be deployed in production contexts without human oversight at multiple stages of the development pipeline. This finding holds even for organisations with sophisticated automated tooling. The feedback-loop security degradation phenomenon [18], the false sense of security effect [8], and the architectural vulnerability spike (322% increase in privilege escalation paths [7]) all point to failure modes that require human architectural and security expertise to detect and prevent. The appropriate framing is therefore not "AI replacing developers" but "AI augmenting developers at well-defined stages, with human expertise providing essential quality control at others."

C. Limitations and Heterogeneity of Evidence

Several limitations constrain the conclusions of this synthesis. First, rapid AI tooling evolution means that findings from studies using 2022–2023 model versions may underestimate the security performance of 2025 frontier models, and vice versa for the feedback-loop degradation findings which used more recent models. Second, many studies use controlled or synthetic settings, with limited longitudinal evidence from genuine production codebases outside of GitClear's dataset. Third, vulnerability measurement methodologies vary substantially across studies—static analysis (CodeQL, Bandit), formal verification (ESBMC), and manual inspection yield different baseline rates. Fourth, the human comparison baseline is difficult to standardise, as human code quality is itself highly variable by developer experience, domain, and organisational context.

D. Implications for the Software Engineering Profession

The findings of this review suggest that the software engineering profession faces a bifurcation risk: AI tools could either elevate all developers to higher productivity with manageable quality trade-offs (if governance frameworks like SAHGF are widely adopted), or create a two-tier profession in which organisations without rigorous AI governance accumulate compounding technical debt and security vulnerabilities while maintaining an illusion of productivity. The 8,000+ AI-assisted startups estimated to require full or partial application rebuilds [26] represent early evidence of the latter trajectory. The emergence of "rescue

engineering" as a nascent professional specialisation is a market signal that warrants serious attention from software engineering educators and standards bodies.

IX. CONCLUSION

This paper has presented the most comprehensive empirical synthesis to date of AI-generated code quality across security, functional correctness, maintainability, and human behaviour dimensions. The evidence base—drawn from 20+ empirical studies, analyses of more than 211 million changed lines of code, controlled experiments, and industry reports spanning 2022–2026—supports the following principal conclusions.

AI coding tools deliver genuine productivity gains: 55% faster task completion, 53.2% higher unit-test pass rates in controlled trials, 84% more successful builds in enterprise deployments, and substantial improvements in developer satisfaction. These gains are real, reproducible, and valuable—but they are not free.

The security costs are substantial and growing. Overall AI-generated code vulnerability rates range from 12% (CodeQLmeasured, GitHub repositories) to 62% (formally verified, across multiple LLMs). Language-specific rates reach 41% in C/C++. AI-generated code is introducing over 10,000 new security findings per month as of June 2025, a 10× acceleration in six months. Iterative AI refinement without human oversight increases critical vulnerabilities by 37.6% in just five iterations. The developer false-sense-of-security effect systematically undermines code review effectiveness.

The maintainability costs are documented and compounding. Refactoring has collapsed from 25% to under 10% of code changes. Code duplication has increased eightfold. Code churn has more than doubled. Delivery stability decreases 7.2% for every 25% increase in AI usage. Technical debt is accumulating at rates that threaten long-term system viability for organisations without active countermeasures.

The solution is not to abandon AI-assisted development—the productivity gains are too substantial and the competitive pressure too intense. The solution is the principled, governance-structured integration of AI tools into development workflows that preserve human expertise at the stages where AI demonstrably fails. The SAHGF framework proposed in this paper provides a seven-stage, evidence-grounded starting point for organisations seeking to operationalise this integration safely. Future research should focus on validating SAHGF in longitudinal production deployments, developing AI-security-awareness training data for next-generation models, and building standardised evaluation frameworks that measure AI code quality holistically across all five dimensions analysed in this paper.

X. FUTURE WORK

This review identifies five high-priority directions for future research. First, longitudinal production deployment studies are needed to validate the SAHGF framework and quantify its efficacy in reducing vulnerability rates and technical debt accumulation in real organisational contexts. Second, security-aware training research should investigate the extent to which incorporating security-annotated training data and RLHF with security-expert feedback can reduce the language-specific vulnerability patterns documented here. Third, benchmark advancement is needed: current benchmarks (HumanEval, SWEBench) do not adequately evaluate security properties of generated code; a new Security-Aware Code Generation Benchmark (SecCGB) would provide a standardised evaluation instrument. Fourth, the cognitive bias dimension—specifically the false-sense-of-security effect and its neurological and educational underpinnings—warrants deeper investigation to inform developer training programmes. Fifth, standardised technical debt measurement frameworks are needed to enable cross-organisational, longitudinal tracking of AI-induced code quality changes with sufficient granularity to guide evidence-based governance decisions.

REFERENCES

- [1] Janki Naik and Prof. Sagar Patel. "Techniques and Issues in Image Mining: Survey." International Journal for Scientific Research and Development 1.2 (2014): 138-141.
- [2] Naik, J., & Patel, S. Tumor Detection and Classification using Decision Tree in Brain MRI.
- [3] Naik Janki Munikumar, & Dr. Pinal J. Patel. (2024). Financial Distress Prediction with Ensemble Machine Learning Models. Utilitas Mathematica, 121, 434–452.
- [4] Stack Overflow, "Stack Overflow Developer Survey 2025," Stack Overflow, 2025. [Online]. Available: <https://survey.stackoverflow.co/>

- [5] J. Novet, "Google says AI writes over 25% of new code at the company," CNBC, 2025; Robison, "Microsoft says AI writes 20–30% of its code," 2024.
- [6] GitHub, "Does GitHub Copilot improve code quality? Here's what the data says," GitHub Blog, Nov. 2024. [Online]. Available: <https://github.blog/>
- [7] Accenture, "Randomized Controlled Trial: GitHub Copilot Impact on Enterprise Developer Productivity," Accenture Research, 2024.
- [8] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in Proc. IEEE S&P, 2022, pp. 754–768; updated in Commun. ACM, vol. 68, no. 2, pp. 96–105, Jan. 2025.
- [9] T. Bisztray et al., "How secure is AI-generated Code: A Large-Scale Comparison of Large Language Models," arXiv:2404.18353, Dec. 2024.
- [10] Apiiro Research, "4x Velocity, 10x Vulnerabilities: AI Coding Assistants Are Shipping More Risks," Apiiro Blog, Sep. 2025. [Online]. Available: <https://apiiro.com/>
- [11] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do Users Write More Insecure Code with AI Assistants?" in Proc. ACM CCS, 2023, pp. 1153–1167.
- [12] GitClear, "AI Copilot Code Quality 2025: 4x Growth in Code Clones," GitClear Research, 2025. [Online]. Available: <https://www.gitclear.com/>
- [13] Google DORA Team, "2024 Accelerate State of DevOps Report," Google Cloud, 2024.
- [14] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.
- [15] Papers with Code, "SWE-Bench Verified Leaderboard," 2025; "LLM Benchmarks 2025," llm-stats.com, 2025.
- [16] C. Jain et al., "LiveCodeBench: Holistic and Contamination Free Evaluation of LLMs for Code," arXiv:2403.07974, 2024.
- [17] X. Liu et al., "EvalPlus / EvoEval: Evolving Coding Benchmarks via LLM," OpenReview, 2024.
- [18] Y. Fu, C. J. Chong, Z. Yao, and I. Neamtiu, "Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study," ACM Trans. Softw. Eng. Methodol., Feb. 2025, doi:10.1145/3716848.
- [19] M. Schreiber and P. Tippe, "Security Vulnerabilities in AI-Generated Code: A Large-Scale Analysis of Public GitHub Repositories," arXiv:2510.26103, Oct. 2025.
- [20] J. Ji, J. Jun, M. Wu, and R. Gelles, "Cybersecurity Risks of AI-Generated Code," Center for Security and Emerging Technology (CSET), Georgetown University, Nov. 2024.
- [21] S. Shukla et al., "Security Degradation in Iterative AI Code Generation: A Systematic Analysis of the Paradox," accepted IEEE-ISTAS 2025, arXiv:2506.11022, 2025.
- [22] Multiple authors, "Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild," arXiv:2603.28592, 2025.
- [23] J. Prather et al., "It's Weird How Little I Know": Developer Perspectives on AI Overreliance, ACM SIGCSE, 2023.
- [24] M. Brown et al., "Trust Differentials in AI Coding Assistance: Junior vs. Senior Developer Acceptance Patterns," in Proc. ACM ICSE, 2024.
- [25] G. Sandoval et al., "Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants," in Proc. USENIX Security, 2023, pp. 2205–2222.
- [26] O. Asare et al., "Is GitHub Copilot a Substitute for Human Pair-Programming? An Empirical Study," in Proc. ICSE SEIP, 2023.
- [27] C. Negri-Ribalta, R. Geraud-Stewart, A. Sergeeva, and G. Lenzini, "A Systematic Literature Review on the Impact of AI Models on the Security of Code Generation," Frontiers Big Data, vol. 7, 2024, doi:10.3389/fdata.2024.1386720.
- [28] Veracode, "2025 GenAI Code Security Report: Comprehensive Data on AI Security Risks and Remediation Strategies," Veracode Research, 2025.
- [29] Gartner Research, "AI-Assisted Development Risk Assessment 2025"; Forrester, "Technical Debt Forecast 2026," 2025.
- [30] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot," arXiv:2302.06590, 2023; GitHub Blog, May 2023.
- [31] Evolution of Programmers' Trust in Generative AI Programming Assistants, arXiv:2509.13253, Sep. 2025.